
Practical No.:	01
Practical Title :	To understand Version Control System / Source Code Management, install git and create a GitHub account.
Aim:	To understand Version Control Systems and install Git, and create an account on GitHub for managing source code.
Objective:	To study Version Control System and Source Code Management concepts. To install Git and create a GitHub account for managing source code.

Theory:**What is a version control system?**

Version control systems are a category of software tools that helps in recording changes made to files by keeping a track of modifications done in the code.

Why Version Control system is so important?

As we know that a software product is developed in collaboration by a group of developers they might be located at different locations and each one of them contributes to some specific kind of functionality/features. So in order to contribute to the product, they made modifications to the source code (either by adding or removing). A version control system is a kind of software that helps the developer team to efficiently communicate and manage (track) all the changes that have been made to the source code along with the information like who made and what changes have been made. A separate branch is created for every contributor who made the changes and the changes aren't merged into the original source code unless all are analyzed as soon as the changes are green signaled they merged to the main source code. It not only keeps source code organized but also improves productivity by making the development process smooth.

Benefits of the version control system:

- Enhances the project development speed by providing efficient collaboration.
- Leverages the productivity, expedites product delivery, and skills of the employees through better communication and assistance.
- Reduce possibilities of errors and conflicts meanwhile project development through traceability to every small change.
- Employees or contributors of the project can contribute from anywhere irrespective of the different geographical locations through this VCS.

Use of Version Control System:

- **A repository:** It can be thought of as a database of changes. It contains all the edits and historical versions (snapshots) of the project.
- **Copy of Work (sometimes called as checkout):** It is the personal copy of all the files in a project. You can edit to this copy, without affecting the work of others and you can finally commit your changes to a repository when you are done making your changes.
- **Working in a group:** Consider yourself working in a company where you are asked to work on some live project. You can't change the main code as it is in production, and any change may cause inconvenience to the user, also you are working in a team so you need to collaborate with your team to and adapt their changes. Version control helps you with

the, merging different requests to main repository without making any undesirable changes.

Types of Version Control Systems:

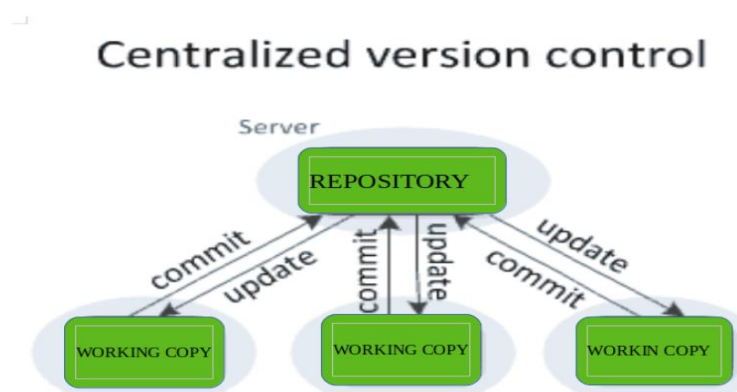
- Local Version Control Systems
- Centralized Version Control Systems
- Distributed Version Control Systems

Local Version Control Systems: It is one of the simplest forms and has a database that kept all the changes to files under revision control. RCS is one of the most common VCS tools. It keeps patch sets (differences between files) in a special format on disk. By adding up all the patches it can then re-create what any file looked like at any point in time.

Centralized Version Control Systems: Centralized version control systems contain just one repository globally and every user need to commit for reflecting one's changes in the repository. It is possible for others to see your changes by updating.

Two things are required to make your changes visible to others which are:

- You commit
- They update



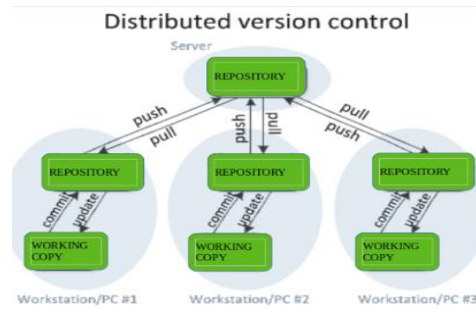
The **benefit** of CVCS (Centralized Version Control Systems) makes collaboration amongst developers along with providing an insight to a certain extent on what everyone else is doing on the project. It allows administrators to fine-grained control over who can do what.

Distributed Version Control Systems: Distributed version control systems contain multiple repositories. Each user has their own repository and working copy. Just committing your changes will not give others access to your changes. This is because commit will reflect those changes in your local repository and you need to push them in order to make them visible on the central repository. Similarly, when you update, you do not get others' changes unless you have first pulled those changes into your repository.

To make your changes visible to others, 4 things are required:

- You commit
- You push
- They pull
- They update

The most popular distributed version control systems are Git, and Mercurial. They help us overcome the problem of single point of failure.



Purpose of Version Control:

- Multiple people can work simultaneously on a single project. Everyone works on and edits their own copy of the files and it is up to them when they wish to share the changes made by them with the rest of the team.
- It also enables one person to use multiple computers to work on a project, so it is valuable even if you are working by yourself.
- It integrates the work that is done simultaneously by different members of the team. In some rare cases, when conflicting edits are made by two people to the same line of a file, then human assistance is requested by the version control system in deciding what should be done.
- Version control provides access to the historical versions of a project. This is insurance against computer crashes or data loss. If any mistake is made, you can easily roll back to a previous version. It is also possible to undo specific edits that too without losing the work done in the meanwhile. It can be easily known when, why, and by whom any part of a file was edited.

Steps for Installing Git for Windows:

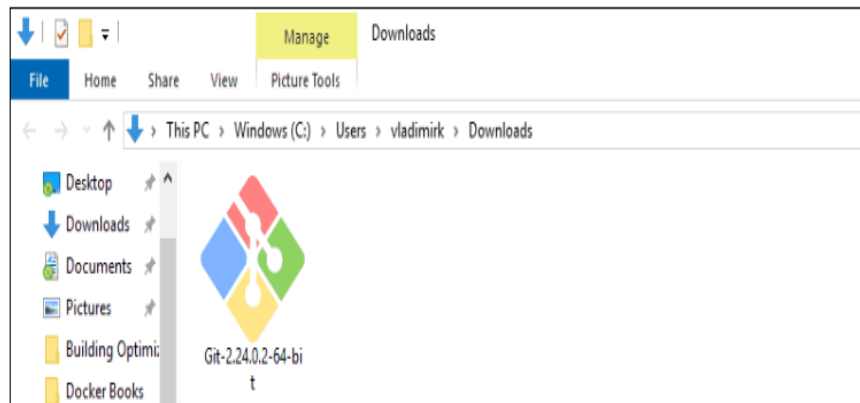
Download Git for Windows

1. Browse to the official Git website: <https://git-scm.com/downloads>
2. Click the download link for Windows and allow the download to complete.

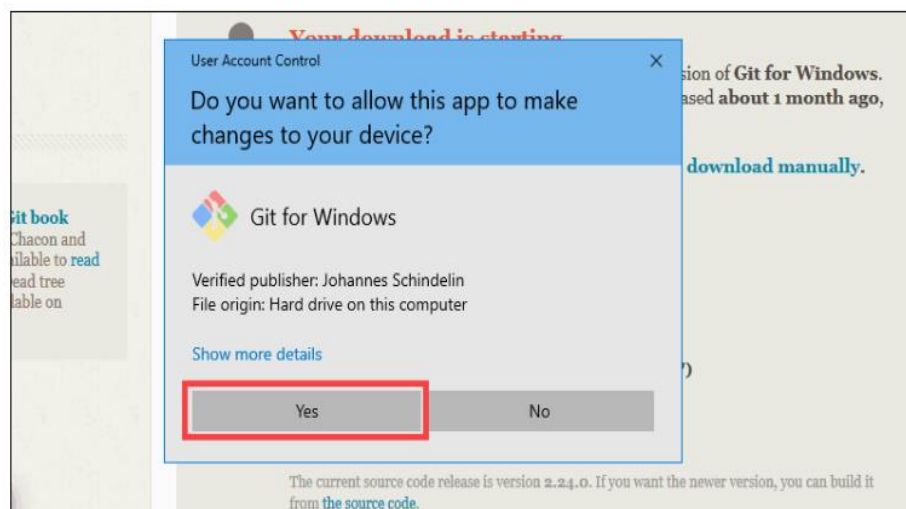


Extract and Launch Git Installer

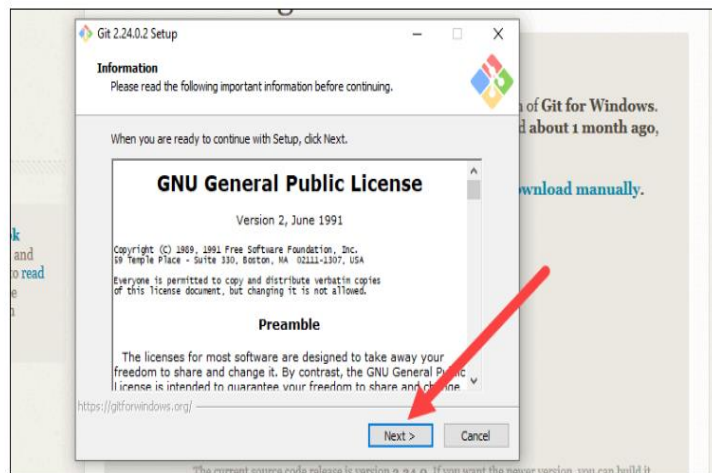
Browse to the download location (or use the download shortcut in your browser). Double-click the file to extract and launch the installer.



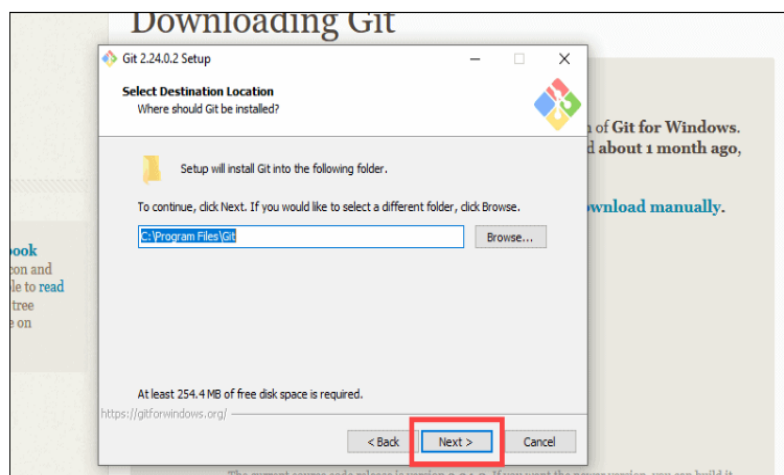
Allow the app to make changes to your device by clicking yes on the User Account Control dialog that opens.



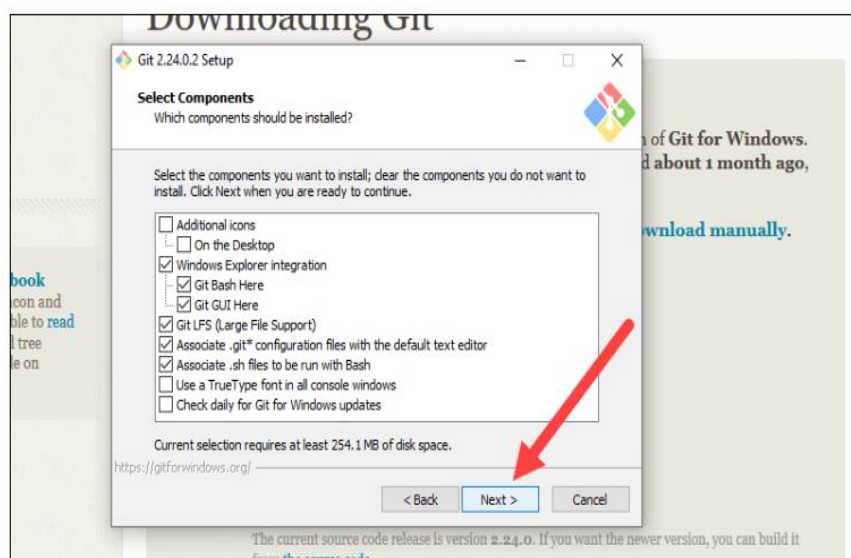
Review the GNU General Public License, and when you're ready to install, click **next**.



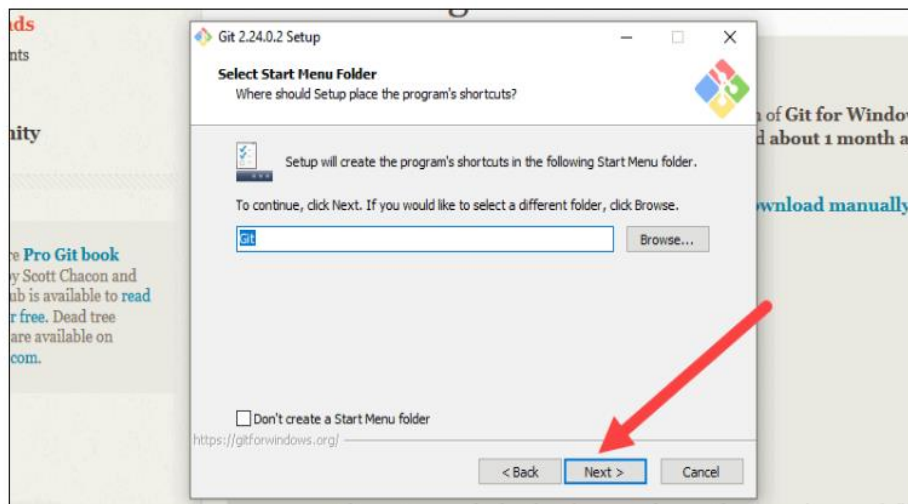
The installer will ask you for an installation location. Leave the default, unless you have reason to change it, and click **next**.



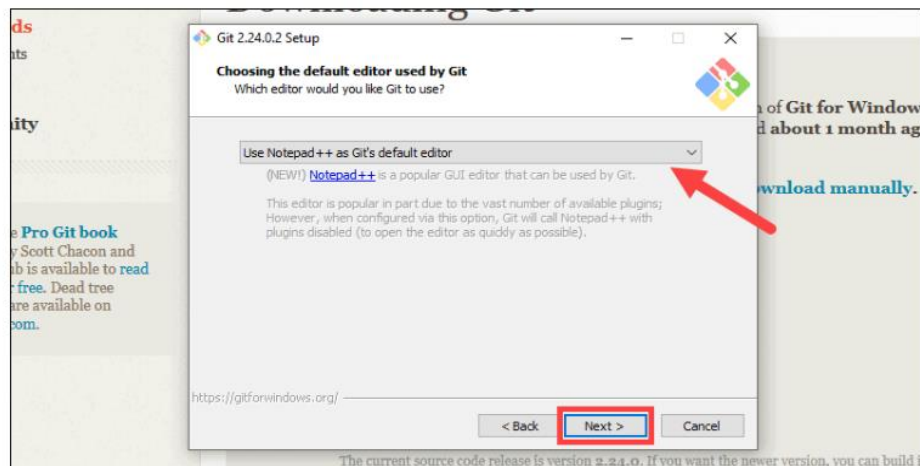
A component selection screen will appear. Leave the defaults unless you have a specific need to change them and click **next**.



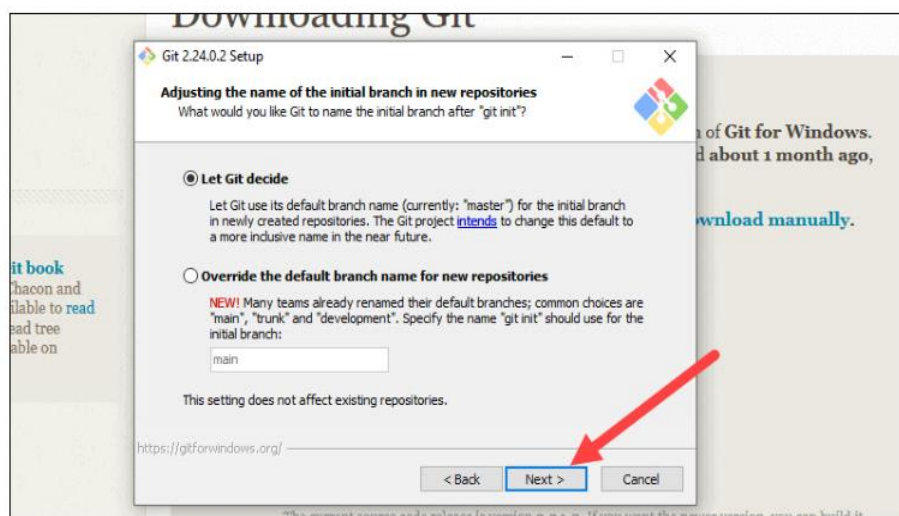
The installer will offer to create a start menu folder. Simply click **next**.



Select a text editor you'd like to use with Git. Use the drop-down menu to select Notepad++ (or whichever text editor you prefer) and click Next

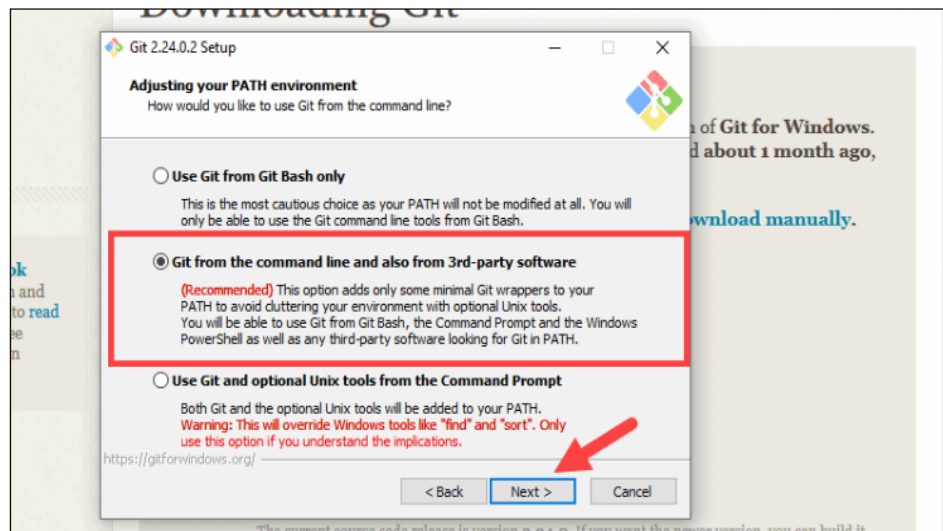


The next step allows you to choose a different name for your initial branch. The default is 'master.' Unless you're working in a team that requires a different name, leave the default option and click next.



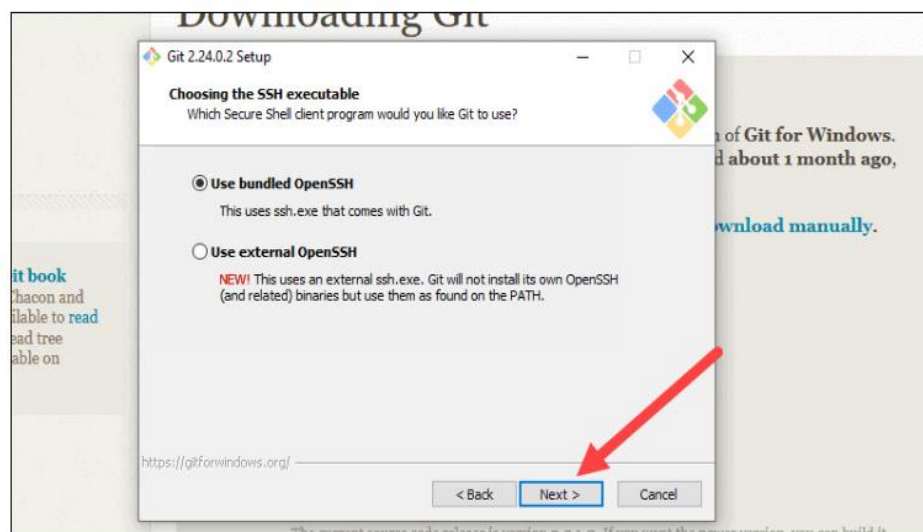
This installation step allows you to change the **PATH** environment. The **PATH** is the default set of

directories included when you run a command from the command line. Leave this on the middle (recommended) selection and click **next**.

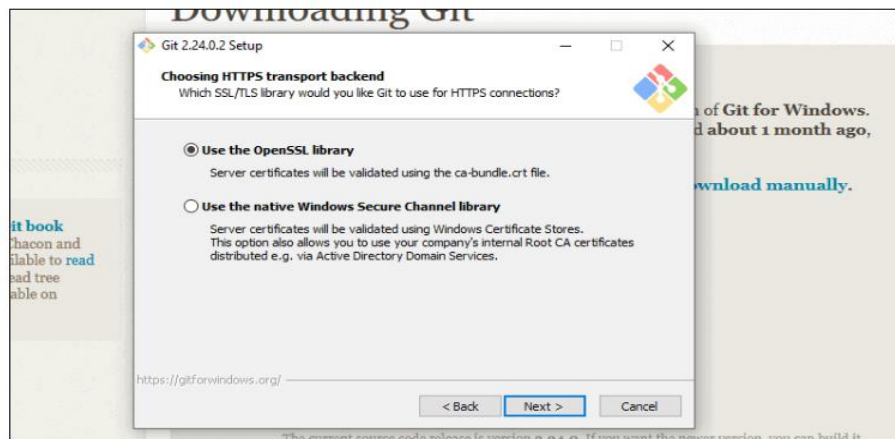


Server Certificates, Line Endings and Terminal Emulators

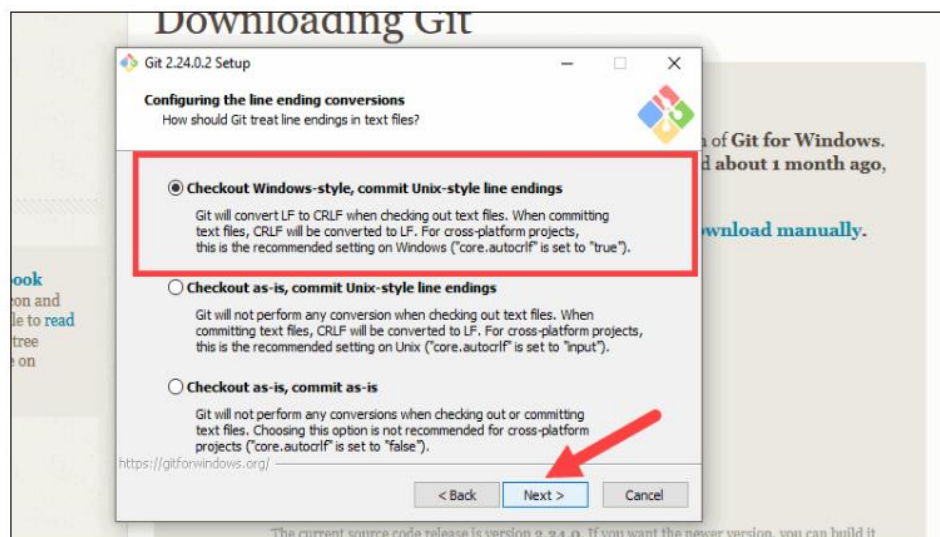
The installer now asks which SSH client you want Git to use. Git already comes with its own SSH client, so if you don't need a specific one, leave the default option and click next.



The next option relates to server certificates. Most users should use the default. If you're working in an Active Directory environment, you may need to switch to Windows Store certificates. Click **Next**.



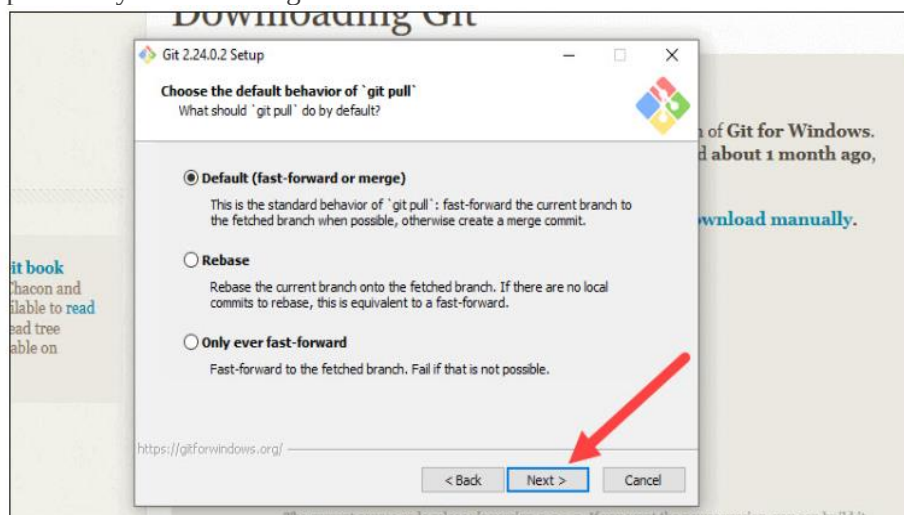
The next selection converts line endings. It is recommended that you leave the default selection. This relates to the way data is formatted and changing this option may cause problems. Click **Next**.



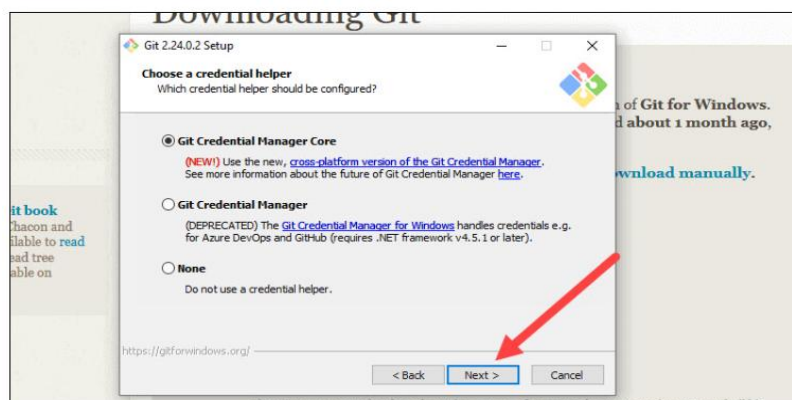
Choose the [terminal emulator](#) you want to use. The default MinTTY is recommended, for its features. Click **Next**.



The installer now asks what the `git pull` command should do. The default option is recommended unless you specifically need to change its behavior. Click **Next** to continue with the installation.

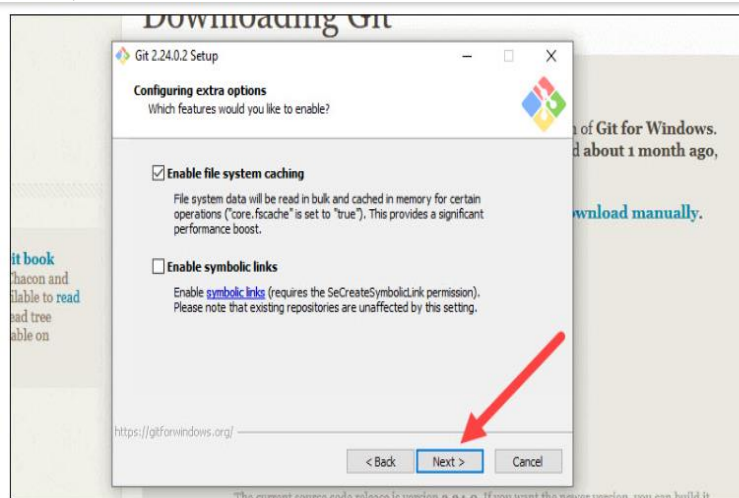


Next you should choose which credential helper to use. Git uses credential helpers to fetch or save credentials. Leave the default option as it is the most stable one, and click **next**.

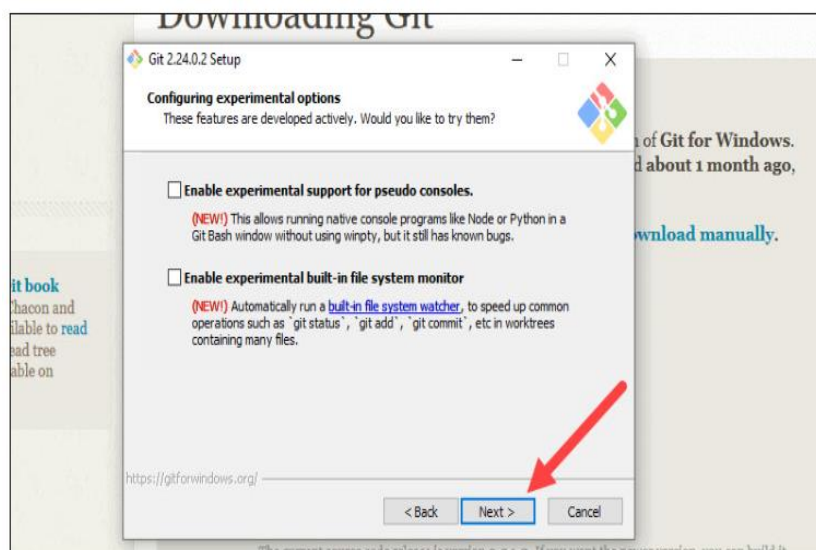


Additional Customization Options

The default options are recommended, however this step allows you to decide which extra option you would like to enable. If you use symbolic links, which are like shortcuts for the command line, tick the box. Click **Next**.

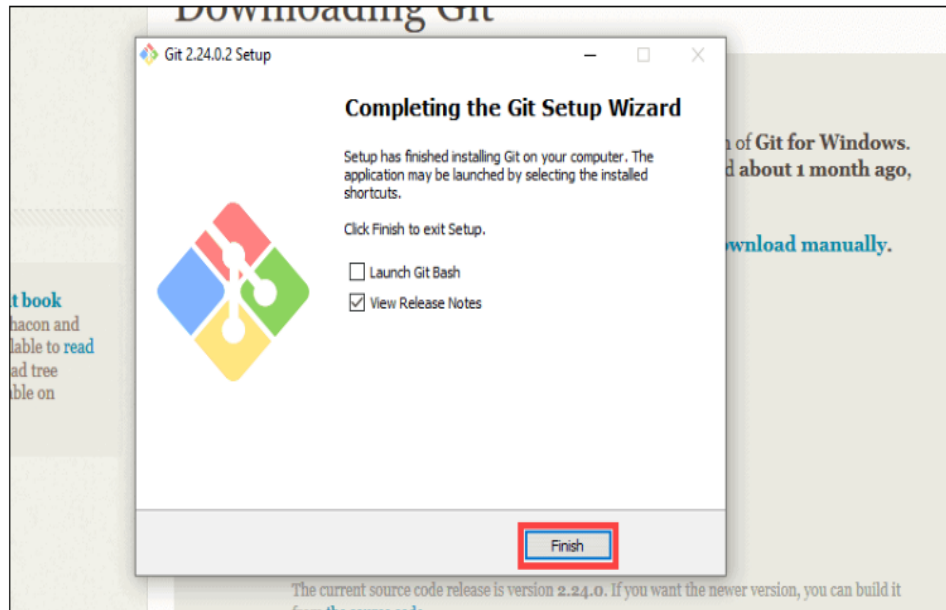


Depending on the version of Git you're installing, it may offer to install experimental features. At the time this article was written, the options to include support for pseudo controls and a built-in file system monitor were offered. Unless you are feeling adventurous, leave them unchecked and click **Install**.



Complete Git Installation Process

Once the installation is complete, tick the boxes to view the Release Notes or Launch Git Bash, then click **Finish**.

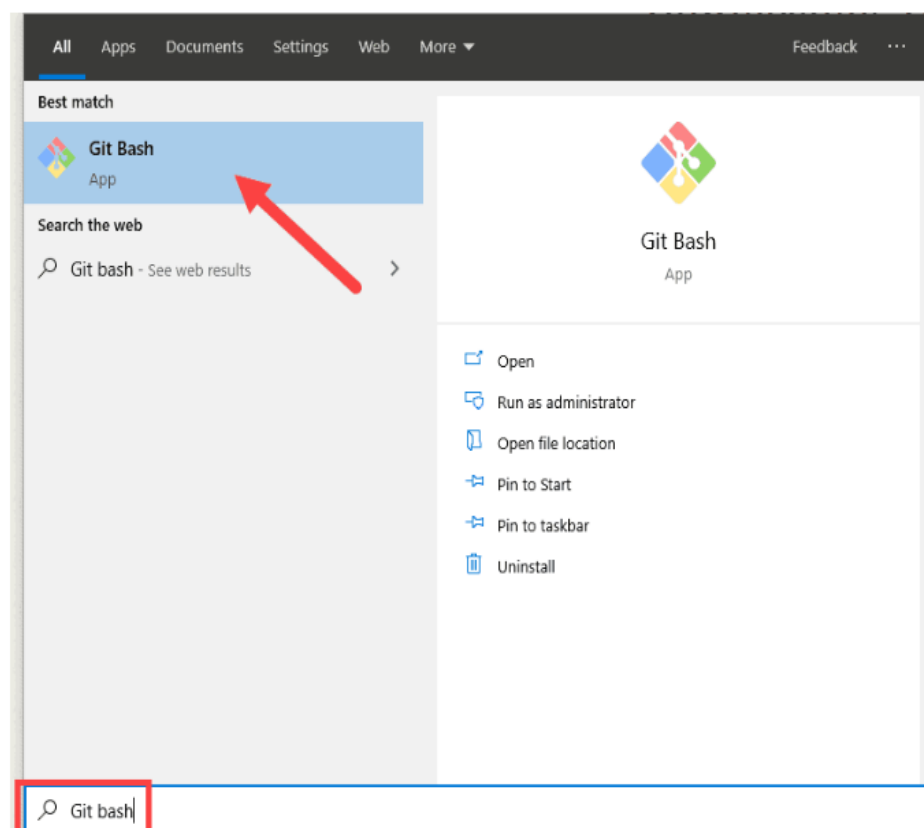


How to Launch Git in Windows

Git has two modes of use – a **bash scripting shell** (or command line) and a **graphical user interface** (GUI).

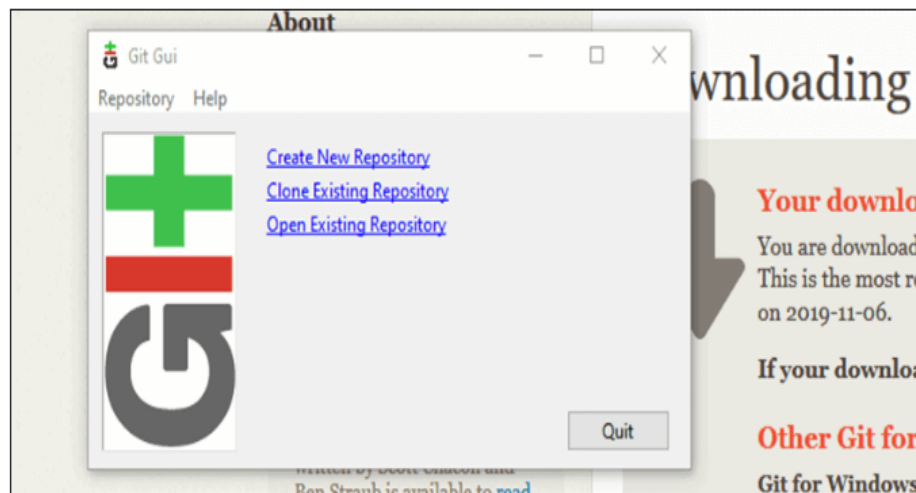
Launch Git Bash Shell

To launch **Git Bash** open the **Windows Start** menu, type *git bash* and press **Enter** (or click the application icon).



Launch Git GUI

To launch **Git GUI** open the **Windows Start** menu, type **git gui** and press **Enter** (or click the application icon).



Set up a GitHub account

You will need a GitHub account to create a GitHub repository where the revision will be stored. If you already have a GitHub account, skip ahead to [Step 2: Create a GitHub repository](#).

1. Go to <https://github.com/join>.
2. Type a user name, your email address, and a password.
3. Choose **Sign up for GitHub**, and then follow the instructions.

Conclusion:

